



GoAAL: an Ontology for Goal-oriented Development of AAL Environments

Marco Cameranesi Università Politecnica delle Marche via Brece Bianche Ancona, Italy m.cameranesi@pm.univpm.it	Claudia Diamantini Università Politecnica delle Marche via Brece Bianche Ancona, Italy c.diamantini@univpm.it
Domenico Potena Università Politecnica delle Marche via Brece Bianche Ancona, Italy d.potena@univpm.it	Emanuele Storti Università Politecnica delle Marche via Brece Bianche Ancona, Italy e.storti@univpm.it

ABSTRACT

Ambient Assisted Living (AAL) systems are playing an important role in the modern society, by helping elderly people to live more independently and support daily activities. Knowledge models have been proposed in the past to describe AAL devices. However, models and methodologies capable to provide assistance to system designers during the development of an AAL environment are still missing. To this aim, in this work we propose an ontology to formally represents all relevant knowledge in the AAL domain ranging from goals to measures and sensors. On its top, a set of logic-based reasoning functions provides advanced support to the development process. In this way, starting from high-level goals the designer can easily retrieve which devices are needed to best meet the project specifications, leading to a cost-effective reduction of development time.

CCS Concepts

- **Information systems** → *Decision support systems*;
- **Compute systems organization** → *Sensor networks*;

Keywords

ambient assisted living, ontology, sensors, logic programming, reasoning

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

SAC 2016, April 04 - 08, 2016, Pisa, Italy

Copyright is held by the owner/author(s). Publication rights licensed to ACM.

ACM 978-1-4503-3739-7/16/04...\$15.00

DOI: <http://dx.doi.org/10.1145/2851613.2851844>

1. INTRODUCTION

During the last years much effort has been made in the field of Ambient Assisted Living (AAL). In particular, many innovative ICT-based solutions have been developed in order to improve quality of life of elderly people, and to enhance their autonomy and safety during daily activities. As a matter of fact, devices for AAL can be easily found in smart homes nowadays, even though in most cases manufacturers rely on their own specifications and standards. Given the huge amount of technological solutions available, this implies for system designers of AAL environments the need to take into account a wide variety of heterogeneous devices, with various characteristics and different performances. Several knowledge-based support systems have been proposed in the past to support the development task, mainly based on ontologies describing sensors and actuators aimed to address issues related to device heterogeneity. However, in this complex and evolving context, a solution capable to take into account the overall development process is still missing.

To address such an issue, in this paper we propose GoAAL, an ontology devoted to support a goal-oriented approach to development of AAL environments. In order to provide assistance to the system designer in choosing the sensors to adopt, the ontology integrates and extends various ontologies focusing on the formal representation of goals of the AAL domain, tasks needed to fulfill them and sensors able to monitor quantitative measures. As a further contribution of the paper, on the top of the ontology we also introduce a reasoning framework based on logic programming and targeted to provide advanced functionalities useful for development. In this way, GoAAL is meant to represent both a reference dictionary for development purposes and a reliable and helpful support tool for AAL system designers in answering to typical questions at design time, at the same time contributing to improve effectiveness and efficiency. Among them, for instance, understanding which sensors are most suitable to fulfill a certain task or a certain high-level goal, together with more complex development issues like the retrieval of the minimal set of sensors capable to provide certain mea-

asures, or the definition of alternative usages for a set of already installed sensors, in case of a scenario with changing requirements.

The rest of the paper is organized as follows: in next Section we discuss the architecture of GoAAL ontology while in Section 3 we define reasoning support services for ontology management and for guiding the development process. Then, in Section 4 we briefly report a case study relying on the ontology and the support services. Section 5 provides a brief review of the state of the art, and finally in Section 6 we draw conclusions and discuss future work.

2. AN ONTOLOGY FOR AAL

Several methodologies for ontology development have been adopted during the last years (see [5] for a survey). In this work we followed an approach based on [8, 11] that starts from the determination of domain and scope of the ontology, which in this case is the formal representation of goals, tasks, measures, sensors and their mutual relations. Next steps, that we report in the following, involve reuse of existing ontologies (Section 2.1), definition of classes and relations aimed at representing the domain of interest (Section 2.2) and implementation, to translate axioms in a machine-readable language (Section 2.3).

2.1 Reuse of existing ontologies

The development of GoAAL has been driven by the analysis of the state of the art of AAL ontologies in order to reuse vocabularies or ontologies that are of particular relevance in the domain. In the following we briefly summarize the main aspects of the external ontologies we selected:

- OntoStar [10] corresponds to the ontological representation of i^* , a framework with an agent-oriented approach for goal-oriented requirement engineering. It provides constructs for representing goals and their decomposition in (sub)goals and tasks.
- DogOnto [1] is an ontology focused on interoperation among systems within a smart environment, comprising device/network independent description of houses, including architectural elements and devices.
- KPIOnto [6] is aimed at describing indicators (PI) for monitoring of performances. PIs correspond to the notion of measures, and are typically classified as either *atomic* or *compound*, depending on the availability of a mathematical formula stating how they can be calculated from other PIs.
- The Semantic Sensor Network (SSN) ontology [3], developed by the W3C Semantic Sensor Networks Incubator Group (SSN-XG), describes capabilities and properties of sensors, the act of sensing and the resulting observations. The ontology is aligned to the DOLCE-Ultralite foundational ontology¹ to further explain concepts and relations.

Prefixes *ois*, *dog*, *kpi* and *ssn* are used in the following to refer to OntoStar, DogOnto, KPIOnto and SSN respectively.

¹<http://www.loa-cnr.it/ontologies/DUL.owl>

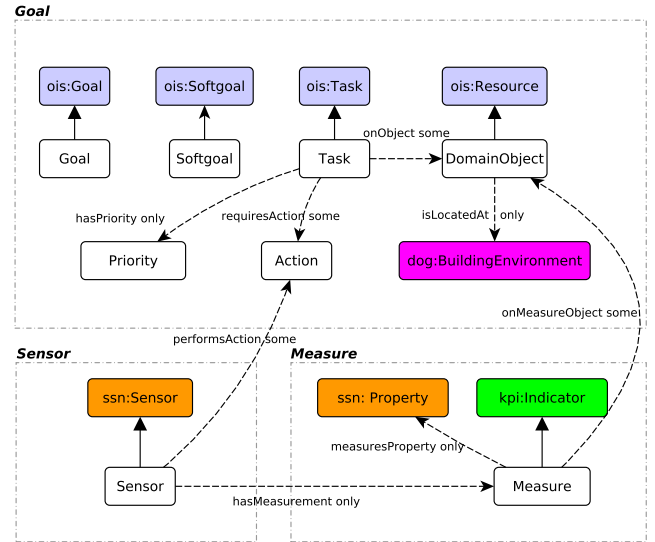


Figure 1: The GoAAL ontology schema.

2.2 GoAAL Architecture

The ontology is conceptually divided in three modules, discussed in next subsections:

- The Goal module describes the goals and their decomposition in subgoals and tasks.
- The Measure module describes measures and their relations with domain objects, to understand which measures can be exploited to accomplish a task.
- The Sensor module is dedicated to define sensors, their properties and capabilities.

The three modules of GoAAL and their interconnections are represented in Figure 1, where the key concepts (white boxes), the extended classes from other ontologies (coloured boxes) and their mutual relations are shown.

2.2.1 Goal module

This module is aimed at modeling goals, soft-goals, tasks and resources. The Goal module is shown in Figure 2 and its classes and relations are explained in detail below:

- **Goal.** This class represents goals ($\sqsubseteq$ ois:Goal) and its decomposition in subgoals and tasks. In our approach, goals are classified according to the degree of emergency of the recognized situation, and named after the codes usually adopted in *triage* for assessing the priority of patients' treatments: critical issues that need to be urgently managed (red), dangerous issues that must be recognized to avoid more serious consequences (yellow) and actions that can improve general health conditions (green). We also refer to a further typology, which covers non-critical services aimed at providing support for comfort and home management (white code). For instance, let us consider a red code goal

related to recognition of environmental issues. Among its subgoals, the *FireDetection* goal can be directly fulfilled through the *FlamesDetection* task to detect the presence of flames in the house. The *HighTemperatureLevelDetection* task contributes as well to the goal, as an overly high temperature in a room may be an evidence for the presence of fire. Furthermore, *SmokeRecognition* subgoal can help to achieve the goal at hand. In turn, this can be fulfilled through tasks *HighLevelOfCO2Detection* or *LowAirTransparencyDetection*.

- **Softgoal.** A softgoal ($\sqsubseteq \text{ois:Softgoal}$) represents conditions for which there is no clear-cut criteria to judge its satisfaction. It is mainly used for non-functional requirements, e.g. it can be important to assure *Usability* of devices in contexts with elderly people.
- **Task.** A task ($\sqsubseteq \text{ois:Task}$) is related to the execution of an action ($\exists \text{requiresAction.Action}$) over a resource ($\exists \text{onObject.DomainObject}$). For instance, the task *MonitorIrregularHeartbeat* requires a *Monitoring* action over *Heartbeat* domain object. A Task may be associated to a priority ($\forall \text{hasPriority.Priority}$).
- **Priority.** The priority of a task (*Highest*, *High*, *AboveNormal*, *Normal*, *BelowNormal*, *Low* and *Lowest*).
- **Action.** The type of action (e.g., *Monitoring*, *Enactment*) that a *Task* can perform over a *DomainObject*.
- **DomainObject.** The representation of the domain objects (e.g., vital parameters like *HeartBeat*, environmental quantities like *Heat* or *CO2*). It is defined as a subclass of Resource in the OntoiStar ontology ($\sqsubseteq \text{ois:Resource}$) and is located in a BuildingEnvironment ($\forall \text{isLocatedAt.dog:BuildingEnvironment}$).
- **dog:BuildingEnvironment.** This class describes all the possible places in a house.

2.2.2 Measure module

The purpose of this module is formalizing measures gathered by sensors. The class Measure extends the class Indicator defined in the KPIOnto ($\sqsubseteq \text{kpi:Indicator}$) and is described in the following together with relevant classes from external ontologies:

- **kpi:Indicator** is the central class of KPIOnto, and its instances describe any metric that can be monitored, e.g. *HeartBeatRate_bpm*, *CO2Level_ppm*. Properties of an indicator are: name, acronym, definition, the aggregation function ($\exists \text{kpi:hasAggrFunction.kpi:AggregationFunction}$), the unit of measurement ($\exists \text{kpi:hasUnitOfMeasurement.kpi:UnitOfMeasurement}$) and a formula ($\forall \text{kpi:hasFormula.kpi:Formula}$).
- **kpi:UnitOfMeasure** represents the symbol and the description of the unit chosen for the indicator, making reference to an external ontology, namely the Measurement Units Ontology².

²<http://idi.fundacionctic.org/muo/muo-vocab.html>

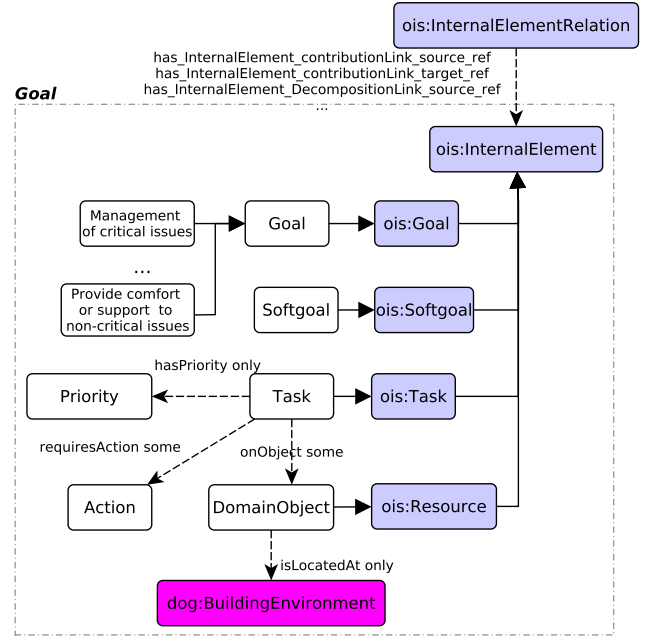


Figure 2: Goal module of the ontology.

- **kpi:AggregationFunction** is the function through which a measure can be aggregated along some dimensions (e.g., the *MeanDailyTemperature_C* is given by averaging temperature values retrieved during the day).
- **kpi:Formula.** It is used to calculate a compound indicator starting from other indicators. Each formula is characterized by its graphical representation, its semantics and the references to its components, which are in turn formulas. For instance, the *BodyMassIndex* is derived by *Weight * Height*.
- **ssn:Property.** This class from SSN ontology describes an observable quality of an event or object. Here, it is related to a given *Measure* ($\forall \text{measuresProperty.ssn:Property}$). For instance, a *CO2Level_ppm* measuresProperty *CO2Level*.
- **MeasureSet**, which describes compound measures that includes more than one submeasure ($\forall \text{itemOf.MeasureSet}$), e.g. *BloodPressure_mmhg* in composed by *DiastolicPressure_mmhg* and *SistolicPressure_mmhg*.

A measure is also related to the specific domain object to which it refers ($\exists \text{hasObject.DomainObject}$).

2.2.3 Sensor module

This module includes a Sensor class which is used to represent all the devices in a smart environment and extends the class Sensor in the SSN ontology ($\sqsubseteq \text{ssn:Sensor}$). The module is depicted in Figure 4.

- **ssn:Sensor.** A sensor in SSN is any entity that can follow a sensing method and observe some property of

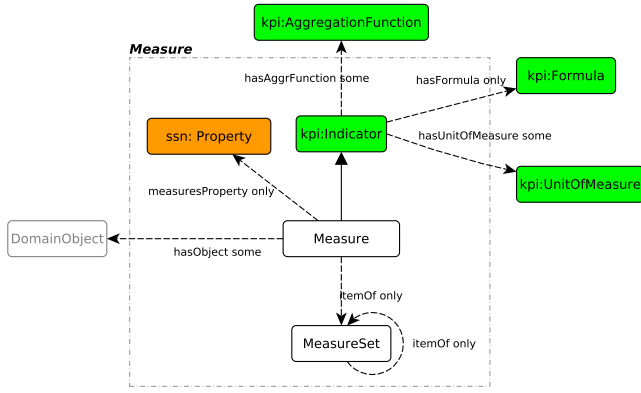


Figure 3: Measure module of the ontology.

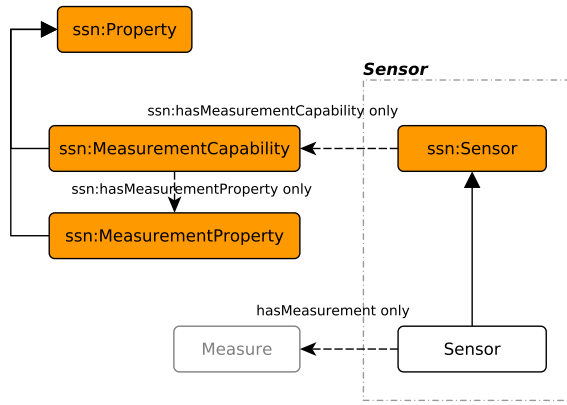


Figure 4: Sensor module of the ontology.

real-world phenomena. Sensors may be physical devices, computational methods or any other thing that can follow a Sensing Method to observe a Property.

- **ssn:MeasurementProperty** is a subclass of **ssn:Property**, which is used to specify technical information about the sensor (e.g. frequency, accuracy, range, precision).
- **ssn:MeasurementCapability** is a subclass of **ssn:Property** that collects together measurement properties and the environmental conditions in which those properties hold. A capability has a measurement property ($\forall \text{ssn:hasMeasurementProperty. MeasurementProperty}$), whereas this last is detailed by specifying the value of the property ($\forall \text{ssn:hasMeasurementPropertyValue. } (\exists \text{ssn:hasQuantityValue})$) and/or, in case the property is associated to a range, the minimum and maximum values ($\forall \text{ssn:hasMeasurementPropertyMinValue. } (\exists \text{ssn:hasQuantityValue})$).

For example, *CO2DetectionSensor*₁₂₃ is manufactured by “ACME₁” and hasMeasurementCapability.frequency of “2/min” and hasMeasurement *CO2Level_in_ppm*.

2.3 Implementation

The GoAAL ontology is implemented in OWL2-RL³ language (based on a subset of OWL-DL), with the exception of the measures’ formulas for which OWL2 does not allow a proper representation. Hence, in order to exploit them for reasoning purposes, these mathematical expressions are stored in a repository relying on the W3C Recommendation MathML⁴ and OpenMath⁵. They provide a language to represent formulas in a machine-readable format with a defined syntax and semantics, which allows to perform automatic manipulation on them.

3. QUERYING AND SUPPORT REASONING FUNCTIONS

The ontological model can be queried through Simple Protocol and RDF Query Language (SPARQL). Possible queries can help to understand which tasks must be performed to achieve a given high-level goal, which measures are devoted to monitor a certain domain object or which sensors are capable to provide a given measure.

More interesting queries on measures’ formulas can be executed, for instance to determine which is the minimal set of sensors providing a given set of measures. However, in this case specific reasoning functions must be defined on the top of the model, by exploiting the formal representation of formulas. In order to define such functionalities it is needed to represent and manipulate formulas within a reasoning framework. To this end, we refer to Logic Programming (LP) as logic-based language to reconcile OWL2-RL and MathML semantics, for its capability to manipulate both (properly translated) OWL2-RL axioms and mathematical equations. Indeed, mathematical formulas are represented as facts by referring to an infix notation like `formula(‘A’, ‘B’+‘C’)`.

In the following, we show some reasoning functions proposed in KPIOnto capable to support generic queries about measures and formulas, that we extended with a set of specific predicates for AAL. We also discuss a set of predicates devoted to the management of consistency in the ontology.

3.1 Reasoning to support development

A set of *basic functionalities* are aimed to enable manipulation of mathematical formulas by extending PRESS (Prolog Equation Solving System)⁶, a formalization of algebra in Prolog that includes predicates for various mathematical tasks, like basic symbolic operations, equation solving and equation rewriting.

More *advanced functionalities* are focused on the extraction of “common measures”: given a set of measures $\phi = \{m_1, m_2, \dots, m_n\}$, common measures of ϕ is the minimal set of measures needed to compute all formulas of ϕ . In [6] such a concept is implemented through the predicate `indToMea( $L_{in}$ ,  $L_{out}$ )`, which takes as input a list L_{in} of measures and generates its common measure set in L_{out} . To give an example, let us consider the following formulas for A ,

³<http://www.w3.org/TR/owl-overview/>

⁴<http://www.w3.org/Math/>

⁵<http://www.openmath.org/>

⁶<http://dream.inf.ed.ac.uk/software/press/>

B and D : `formula('A', 'B'+ 'C')`, `formula('B', 'D'+ 'E')`, and `formula('D', 'F'+ 'G')`. If $\phi = \{A, B, C, D, E, F, G\}$, then a solution for `indToMea`(ϕ, L_{out}) is $L_{out} = \{C, D, E, G\}$. In fact, with D and G we can compute F by reverting the third formula. Then, having D and E , B is obtained according to the second formula. Finally, using the first formula A can be derived from B and C . Let us note that other possible minimal solutions exists, like $\{A, B, D, G\}$ and $\{B, C, F, G\}$. The predicate `meaToInd`(L_{in}, L_{out}) implements the inverse of `indToMea`: given a set of available measures L_{in} , the predicate returns in L_{out} all those measures that are derivable from them, e.g. if $L_{in} = \{B, C, E\}$ it is possible to derive $L_{out} = \{A, B, C, D, E\}$.

We extended this set of predicates by defining the following ones, to specifically support the retrieval of information in GoAAL:

- `meaToSen`(L_m, L_s): given a set L_m of measures, returns the sets L_s of sensors needed to compute them. This relies on determining from `indToMea` the minimum set of measures actually needed, and then on the execution of a Prolog goal in the form `hasMeasure(X, measure)` for each measure. For example, let us consider a knowledge base with sensors $s1$, $s2$ and $s3$ and measures A , B and C :

```

- hasMeasure('s1', 'A'). hasMeasure('s1', 'C').
- hasMeasure('s2', 'B').
- hasMeasure('s3', 'B'). hasMeasure('s3', 'C').

```

In such a case, given $L_m = \{A, B, C\}$, then $L_s = \{s1, s2, s3\}, \{s1, s2\}, \{s1, s3\}$.

- `senToMea`(L_s, L_m) given a set L_s of sensors, returns the set L_m of measures that can be calculated: at first the list of available measures is retrieved by exploiting the goal `hasMeasure(sensor, X)` for each sensor in L_s . Then, `meaToInd` is executed to expand the list of obtained computable measures.
- `taskToSen`(L_t, L_s) given a set L_t of tasks, returns all the sensors L_s capable to achieve them. At first the set of domain objects related to the tasks are retrieved from the ontology, and in the same way all the measures linked to such objects are found. Then, predicate `meaToSen` is executed to retrieve the sensors.

3.2 Consistency management

In the context of this work, the ontology is meant to be populated before the beginning of the development phase. Usually this happens in an incremental way, as system designers reuse already existing definitions of sensors and measures for following projects. A typical problem in collaborative and incremental knowledge bases is consistency management. This can be usually achieved by evaluating the (in)coherence of a new instance with respect to the ontology schema, by exploiting a consistency service available in OWL reasoners.

For what concerns formulas, a check is needed to verify whether a new added formula is consistent with the already existing formulas, from a logical-mathematical point of view.

On the contrary, the answer to a query could produce inconsistent results. In this case the consistency check is implemented as a Prolog predicate according to the following definition. Let $\mathcal{KB}$ be a knowledge base containing formulas. Given a new measure m_i , its formula f_i is consistent with $\mathcal{KB}$ iff (a) f_i is unique in the ontology (there is no identical formula), (b) its formula is not equivalent to a formula already in $\mathcal{KB}$ and (c) its formula must not contradict any other already defined formula.

As for the first condition (a), the `identical(Equation, X, X=Solution)` predicate is introduced to avoid to insert twice the same definition in the ontology. The reasoner firstly rewrites the `Equation` so that the variable X is only on the left side. Then it searches the whole theory for a formula having the same structure of the rewritten `Equation`. Moreover, (b) during ontology population, it is useful to individuate and manage duplications, i.e. given two formulas f_1 and f_2 , they are equivalent if f_1 can be rewritten as f_2 . The predicate `equivalence(Equation, X, Y)` implements this behaviour by relying on functionalities for formula manipulation, to find formulas Y that are equivalent to $X=Equation$. Finally, (c) we defined a predicate `incoherence(Equation, X, Y=S)` useful to verify that when a formula $X=Equation$ is added or existing formulas are updated or deleted, the change do not contradict the ontology. A formula f_2 for a new measure m_2 is coherent with the ontology if, whenever there exists a formula f_1 for a measure m_1 already defined in the KPIOnto, such that f_2 can be rewritten as f_1 , then the equivalence $m_1 \equiv m_2$ does not contradict the ontology. Otherwise we have an incoherence.

We refer the interested reader to our previous work [6] for further details about manipulation of formulas and consistency management in ontologies.

4. APPLICATIONS

The GoAAL ontology and the reasoning functionalities described in this paper have been developed within a collaborative project called HiCMO [9], that lasted from 2011 to 2015. The high-level goal of the project was to build a platform providing advanced services for monitoring domestic activities to ensure the wellness of elderly people. Interoperability among devices is achieved by representing each device as a software module, that is semantically lifted through concepts in GoAAL. Among them, environmental sensors (e.g., to monitor temperature, humidity, pressure) and wearable sensors integrated in clothes and accessories (to reveals information about the user stress level or the presence of the user in specific areas of the house). Other more advanced sensors include telemedicine systems to monitor vital parameters of the users and smart kitchens that integrate sensors to reveal events related to the user activities.

In the context of the project, two scenarios supported by the ontology and the reasoning functionalities are the development from scratch of an AAL system, starting from user requirements expressed as GoAAL high-level goals, or the redesign of some functionalities already offered by the system in order to meet new specifications, e.g. because the patient is under treatment. For lack of space we briefly show how the ontology can be exploited for the second one: at first,

by following the decomposition of the new goal, the set of requested DomainObjects and the related measures M_r are retrieved. Let us define the set of sensors already available by S . By execution of predicate `senToMea(S,M)` the set M of measures provided by S is retrieved. In case the new requested measures are included in M (i.e., $M_r \subseteq M$), it means that they are already available. Hence, the system designer can reuse the sensors according to the new specifications. On the contrary, if $M_r \not\subseteq M$ it means that some requested measures in M_r are not available. Let us then call such a set $M'_r = M_r \setminus (M \cap M_r)$. In order to solve the problem in a cost-effective way, we have to find the possible sets S' of sensors needed to compute M'_r . This can be achieved by referring to the previously discussed predicate `meaToSen(M',S')`. Finally, we can further filter results in S' in order to select the set of sensors with minimal cardinality or having an overall minimum price.

5. RELATED WORK

In the last decade, researchers made many efforts in the definition of knowledge models to represent relevant concepts in the field of Ambient Assisted Living (AAL), for a variety of support tasks ranging from integration of heterogeneous devices to monitoring/analysis of sensor networks.

The notion of *goal* has been widely adopted in several areas of computer science and many approaches for goal modeling have been proposed in the Literature. Among the methodologies, KAOS (Knowledge Acquisition on automated Specification [4]) introduced a formal framework comprising AND and OR decompositions between goals describing desired states over entities, achieved by actions assigned to agents.

For what concerns *sensor* network ontologies, in the SOUPA Project [2] authors define a set of ontologies focused on describing places, persons and intentions, to support applications in the field of ubiquitous computing within a smart environment. DomoML [14] is an ontology aimed at defining a standard language that allows the integration of products and services coming from different manufacturers. Other improvements are introduced by the DogOnt ontology [1], that extends the DomoML functionalities, in which each object is classified according to its controllability. A relevant contribution comes also from the W3C Semantic Sensor Networks Incubator group. In their work [3], they highlight the importance of semantic technologies in order to manage and query data coming from a great number of heterogeneous sensors.

Measure modeling and analysis have been widely studied in the field of Performance Management in enterprises, where performance indicators (PI) are usually defined in a multi-dimensional context. Formal models for measures are introduced in [12], with a goal-oriented meta-modeling framework where goals are defined by means of PIs, both represented by referring to a logical language, while in [13] the Goal-oriented Requirement Language is enriched with the concept of PIs. In a previous work of ours [7], we explicitly took into account inference mechanisms and formula manipulation for indicators, that are not enabled by most of other works. To the best of our knowledge, there is also no prior work in the AAL field on the exploitation of formal reasoning on the

structure of mathematical formulas and only a few works explicitly consider the notion of formula.

6. CONCLUSION

In this work, we propose an ontology describing the AAL domain in terms of goals, tasks, measures and sensors, to support the goal-oriented development process for AAL systems. A reasoning framework based on logic programming aims to retrieve information and to maintain the ontology consistent. As a by-product, the ontology provides also a reusable, incrementally built knowledge base for reference.

Besides the applications shown in the paper, further usages of GoAAL include context-aware systems for an automated home, in which a sensor network continuously monitors relevant measures that are stored in a repository. Here, the ontology provides a reference vocabulary to semantically enrich relevant terms of the domain (i.e. sensors, measures, rooms). From an analytical perspective, it can also be used in an ontology-based data access (OBDA) approach, to make queries over the repository using ontology terms and exploiting reasoning functionalities.

7. REFERENCES

- [1] D. Bonino and F. Corno. DogOnt - ontology modeling for intelligent domestic environments. In *Proceedings of the 7th International Conference on The Semantic Web, ISWC '08*, pages 790–803, Berlin, Heidelberg, 2008. Springer-Verlag.
- [2] H. Chen, T. Finin, and A. Joshi. The soupa ontology for pervasive computing. In V. Tamma, S. Cranefield, T. Finin, and S. Willmott, editors, *Ontologies for Agents: Theory and Experiences*, Whitestein Series in Software Agent Technologies, pages 233–258. Birkhäuser Basel, 2005.
- [3] Compton, M. et al. The SSN Ontology of the W3C Semantic Sensor Network Incubator Group. *Web Semantics: Science, Services and Agents on the World Wide Web*, 17(0), 2012.
- [4] A. Dardenne, A. van Lamsweerde, and S. Fickas. Goal-directed requirements acquisition. *Science of Computer Programming*, 20(1- 2):3 – 50, 1993.
- [5] M. Darlington and S. Culley. Investigating ontology development for engineering design support. *Advanced Engineering Informatics*, 22(1):112 – 134, 2008. Intelligent computing in engineering and architecture.
- [6] C. Diamantini, D. Potena, and E. Storti. SemPI: A semantic framework for the collaborative construction and maintenance of a shared dictionary of performance indicators. *Future Generation Computer Systems*, 54:352–365, 2015.
- [7] C. Diamantini, D. Potena, E. Storti, and H. Zhang. An ontology-based data exploration tool for key performance indicators. In R. Meersman, H. Panetto, T. Dillon, M. Missikoff, L. Liu, O. Pastor, A. Cuzzocrea, and T. Sellis, editors, *On the Move to Meaningful Internet Systems: OTM 2014 Conferences*, volume 8841 of *Lecture Notes in Computer Science*, pages 727–744. Springer Berlin Heidelberg, 2014.

- [8] M. Fernandez, A. Perez, and N. Juristo. METHONTOLOGY: from Ontological Art towards Ontological Engineering. In *Proc. of the AAAI97 Spring Symposium Series on Ontological Engineering*, pages 33–40, Stanford, USA, March 1997.
- [9] L. Rossi et al. Interoperability issues among smart home technological frameworks. In *Mechatronic and Embedded Systems and Applications (MESA), 2014 IEEE/ASME 10th International Conference on*, pages 1–7, Sept 2014.
- [10] K. Najera, A. Perini, A. Martínez, and H. Estrada. Supporting *i** model integration through an ontology-based approach. In *Proceedings of the 5th International i* Workshop 2011, Trento, Italy, August 28-29, 2011*, pages 43–48, 2011.
- [11] N. F. Noy and D. L. McGuinness. Ontology development 101: A guide to creating your first ontology. *Development*, 32(1):1–25, 2001.
- [12] V. Popova and V. Sharpanskykh. Formal modelling of organisational goals based on performance indicators. *Data & Knowledge Engineering*, 70(4):335–364, 2011.
- [13] A. Pourshahid, G. Richards, and D. Amyot. Toward a goal-oriented, business intelligence decision-making framework. In G. Babin, K. Stanoevska-Slabeva, and P. Kropf, editors, *MCETECH*, volume 78 of *Lecture Notes in Business Information Processing*, pages 100–115. Springer, 2011.
- [14] L. Sommaruga, A. Perri, and F. Furfari. DomoML-env: an ontology for human home interaction. In P. Bouquet and G. Tummarello, editors, *SWAP*, volume 166 of *CEUR Workshop Proceedings*. CEUR-WS.org, 2005.